uc3m | Universidad Carlos III de Madrid

Máster Universitario en Ciencia y Tecnología Informática

Proyecto final. Inteligencia artificial con inspiración biológica

“Optimización de conjuntos de datos para Machine Learning”

Yulicenia Díaz Cabrera

Leganés (Madrid), España. Diciembre 2025.

RESUMEN

Este trabajo aborda la mejora de un modelo lineal cuando los datos presentan relaciones no estrictamente lineales. El problema es que un lineal puro pierde precisión si hay curvaturas o interacciones. El objetivo fue construir una transformación de entrada $v(X)$ mediante búsqueda evolutiva y entrenar después una regresión lineal regularizada (RidgeCV), comparándola únicamente contra su línea base lineal en el mismo conjunto de datos. Se siguió un proceso reproducible con hold-out de validación, escalado y modelo en un pipeline, y una biblioteca de transformaciones suaves con raíz, logaritmo, potencias, recíproco en dominios válidos, trigonométricas normalizadas e interacciones numéricas. La búsqueda evolutiva selecciona y sintetiza atributos con un criterio de MAE. En dos conjuntos de regresión se observó una ganancia consistente respecto al lineal base, en California Housing el MAE bajó de 0.533 a 0.438 (mejora relativa 18 %), en Airfoil Self-Noise descendió de 3.672 a 2.962 (19 %). Como referencia no obligatoria del proyecto, se reportaron además resultados de Random Forest y k-vecinos para situar el nivel de dificultad del problema, sin formar parte del objetivo de optimización. La aportación es un procedimiento simple y reproducible para exprimir un modelo lineal con técnicas bio-inspiradas, manteniendo interpretabilidad y coste bajo y obteniendo mejoras frente a su propia línea base.

Palabras clave: optimización de datos; ingeniería de características; modelos lineales; algoritmos evolutivos; validación.

ÍNDICE GENERAL

1. INTRODUCCIÓN. 1
2. MARCO TEÓRICO. 3
3. METODOLOGÍA 5
4. RESULTADOS 7
5. CONCLUSIONES 10
BIBLIOGRAFÍA 11
6. ANEXOS. 12

ÍNDICE DE FIGURAS

1 Dashboard California 7
2 Curva de fitness California 8
3 Dashboard Airfoil Self-Noise 9

ÍNDICE DE TABLAS

1 Resultados en *California*. 8
2 Resultados en *Airfoil Self-Noise*. 9

1. INTRODUCCIÓN

El aprendizaje supervisado busca construir un modelo que, a partir de un conjunto de atributos de entrada, produzca una predicción con bajo error respecto a los valores reales. En este contexto, el rendimiento final no depende sólo del algoritmo de regresión elegido, sino también de la representación de los datos con la que dicho algoritmo aprende. Por ello, la optimización del conjunto de datos mediante síntesis de atributos y/o reducción de dimensionalidad, se plantea como una vía práctica para mejorar la capacidad predictiva sin cambiar el modelo de base.

El problema que se aborda en este trabajo consiste en diseñar un procedimiento $v(\cdot)$ que transforme las matrices de atributos de entrenamiento y validación en representaciones mejor adaptadas al aprendizaje. La calidad de la transformación se evalúa entrenando un mismo modelo de regresión sobre la representación original y sobre la representación optimizada, y comparando el error medio absoluto (MAE) en validación. El sistema se considera exitoso si, con la misma configuración del modelo, el MAE en la versión optimizada es menor que en la versión original. Esta formulación deriva del enunciado oficial de la práctica, que establece los conjuntos (X_t, y_t) y (X_v, y_v), la construcción de $X'_t = v(X_t)$ y $X'_v = v(X_v)$, y el criterio de éxito basado en el error absoluto en validación.

El contexto general que motiva este trabajo es doble. Por un lado, en problemas reales de regresión es habitual encontrar relaciones no lineales o interacciones entre variables que un modelo lineal simple no captura de forma directa. Por otro, reducir dimensionalidad puede bajar el ruido y estabilizar el ajuste, lo que ayuda a controlar el sobreajuste y a mejorar la generalización. Por ello, resulta útil contar con un mecanismo automático que explore transformaciones de las variables, seleccione subconjuntos informativos y entregue una matriz de atributos optimizada para entrenar después un modelo sencillo y trazable.

La propuesta se enmarca dentro de la computación evolutiva, se exploran transformaciones y selecciones de atributos con un algoritmo de búsqueda poblacional y se adopta el MAE en validación como función de fitness. De este modo, la optimización del conjunto de datos queda desacoplada del modelo final, pero su mejora se mide con el propio desempeño del modelo, que es lo que interesa en práctica.

La estructura del documento es la siguiente. En la Metodología se describe la construcción del espacio de atributos candidatos, la función objetivo, y los elementos del algoritmo evolutivo empleados para explorar dicho espacio. En Resultados se presentan las métricas obtenidas y las visualizaciones de dispersión y curvas de fitness, comparando la representación optimizada frente a la base. Finalmente, en Conclusiones se resumen los hallazgos principales y se señalan límites y líneas de mejora.

Objetivo. Desarrollar y evaluar un sistema $v(\cdot)$ capaz de transformar un conjunto de datos de regresión de modo que, manteniendo fijo el modelo de aprendizaje, se reduzca el error en validación respecto a la representación original, siguiendo el criterio de éxito fijado en el enunciado de la práctica.

2. MARCO TEÓRICO

La calidad de un sistema de aprendizaje automático depende en gran medida de la forma en que los datos se preparan, organizan y representan. La literatura subraya que la gestión de datos como limpieza, integración, y diseño de atributos determina tanto el rendimiento como la reproducibilidad de los modelos, y que una fracción importante del beneficio práctico proviene de decisiones previas al entrenamiento más que del modelo mismo [1]. En este sentido, la ingeniería de atributos busca transformar el espacio original de variables en una representación que haga más lineales o separables las relaciones subyacentes sin recurrir necesariamente a arquitecturas muy complejas.

Entre las transformaciones elementales más usadas se encuentran las raíces cuadradas, los logaritmos estables (por ejemplo, $\log(1 + x)$), las potencias (x^2, x^3) y los recíprocos definidos en dominios válidos. Estas operaciones ayudan a estabilizar varianzas, reducir asimetrías y comprimir colas pesadas, lo que facilita el ajuste posterior. De forma complementaria, la codificación one–hot en variables categóricas de baja cardinalidad permite introducir señales discretas sin imponer un orden artificial, mientras que las interacciones multiplicativas entre predictores numéricos exponen efectos combinados que a menudo son relevantes en fenómenos reales. La idea general es construir una familia de atributos sintéticos $v(X)$ que aumente la capacidad expresiva con transformaciones interpretables y controladas.

Los modelos lineales generalizados y, en particular, la regresión lineal con regularización L_2 (Ridge), se benefician de estas expansiones del espacio de entrada: con una representación adecuada, un estimador lineal puede aproximar vínculos no lineales con buena estabilidad numérica. La regularización actúa como mecanismo de control de complejidad ante colinealidad y crecimiento del número de atributos, y está bien estudiada en escenarios donde el aprendizaje se realiza sobre datos normalizados o potencialmente normalizados respecto a su esquema de origen [2]. Esta combinación de transformaciones simples + modelo lineal regularizado, ofrece interpretabilidad, coste computacional moderado y una base sólida para comparar contra métodos no lineales.

La selección de atributos, ya sea entre variables originales o sintetizadas, plantea un problema combinatorio desafiante. Los métodos evolutivos proporcionan una alternativa robusta cuando el espacio de búsqueda es grande, no convexo y con múltiples óptimos locales. Los algoritmos genéticos y las estrategias evolutivas operan sobre representaciones discretas o continuas aplicando operadores de variación (mutación, recombinación) y selección basados en una función de aptitud, sin requerir derivadas y con tolerancia al ruido. En el ámbito de aprendizaje automático, estas metaheurísticas se han utilizado para seleccionar subconjuntos de atributos, ajustar hiperparámetros y, en general, para explorar configuraciones que mejoran la generalización bajo restricciones de coste [3].

Finalmente, la evaluación comparativa suele apoyarse en modelos base con distintos

sesgos inductivos. Métodos de vecinos cercanos (KNN) capturan relaciones locales y no paramétricas mediante distancias en el espacio transformado, mientras que los ensambles de Random Forest modelan particiones no lineales mediante agregación de múltiples árboles con muestreo y aleatoriedad en atributos. La comparación frente a estimadores lineales regularizados permite valorar el efecto real de la representación de datos y cuando la ingeniería de atributos es efectiva, los modelos lineales pueden acercarse al desempeño de alternativas más flexibles, manteniendo interpretabilidad y simplicidad.

3. METODOLOGÍA

Se adoptó un flujo de trabajo simple y reproducible, con la preparación de datos, generación de atributos candidatos, búsqueda evolutiva sobre máscaras de selección y evaluación comparativa con métricas estándar. El conjunto se dividió en entrenamiento y validación mediante holdout 80/20 con semilla fija (42), de modo que todas las comparaciones se apoyan en el mismo split y no dependen del azar de muestreo.

Datos. Se emplearon dos conjuntos. (i) Un conjunto estilo "California housing", donde la variable objetivo es el valor medio de vivienda (`MedHouseVal`), es un problema de regresión tabular con predominio de variables numéricas. (ii) El *Airfoil Self-Noise Dataset* de NASA (UCI/Kaggle), donde la variable objetivo es el nivel de presión sonora escalado SSPL y los predictores incluyen frecuencia, ángulo de ataque, cuerda, velocidad del flujo libre y espesor de desplazamiento por succión. En variables categóricas de baja cardinalidad (si las hubiese), se aplicó one–hot con un tope razonable de niveles para no inflar la dimensionalidad.

Espacio de atributos. Se trabajó con un catálogo de transformaciones elementales sobre predictores numéricos: $\sqrt{\cdot}$ solo en dominios no negativos, $\log(1 + x)$ para estabilizar varianza en colas largas, potencias x^2, x^3 para curvatura suave y el recíproco $1/x$ en dominios positivos. Además, se generaron interacciones multiplicativas entre pares de variables numéricas restringidas a las de mayor varianza en entrenamiento (top ≈ 8) para contener el tamaño del espacio de búsqueda. Las dummies, entraron como atributos sin transformaciones. Antes de evaluar un conjunto de atributos, se estandarizó internamente para estabilizar el ajuste del modelo.

Modelo base y función de aptitud. La aptitud de cada máscara de atributos se midió entrenando un modelo lineal con regularización L_2 Ridge y, en paralelo, una regresión lineal pura, se retuvo automáticamente la de menor MAE en validación para penalizar combinaciones que indujesen colinealidad o varianza alta. La métrica de optimización fue Mean Absolute Error (MAE) por su robustez ante atípicos. Para reportar y comparar se añadieron Root Mean Squared Error (RMSE) y R^2.

Búsqueda evolutiva. Se utilizó una estrategia evolutiva $1+\lambda$ sobre máscaras binarias, donde cada individuo es un vector $\{0, 1\}^d$ que indica qué atributos de $v(X)$ se incluyen. La mutación consistió en voltear bits con probabilidad baja y en forzar al menos un atributo activo. En cada generación se evaluaron $\lambda = 8$ descendientes más el progenitor (9 individuos por generación) y se seleccionó el mejor si mejoraba el MAE. Se incorporó parada temprana por tolerancia y paciencia para evitar gasto de cómputo cuando no hubiese progreso sustantivo y se registró la fitness curve (MAE vs. generación) como diagnóstico de convergencia y de posibles óptimos locales. Con esta configuración se consiguió un equilibrio razonable entre exploración por mutación y selección estricta del mejor.

Parámetros evolutivos y tamaño del cromosoma. En términos de evaluaciones, con 120 generaciones y 1+8 individuos por generación, se realizaron del orden de 1080 evaluaciones por ejecución. La longitud del cromosoma coincide con el número de atributos candidatos generados:

$$\text{longitud} \;=\; (\#\text{numéricas}) \times 6 \;+\; \binom{k}{2} \;+\; \#\text{dummies},$$

donde las 6 transformaciones univariadas consideradas fueron *id*, $\sqrt{\cdot}$, $\log(1 + x)$, x^2, x^3 e *inv*; las interacciones multiplicativas se limitaron a las k columnas numéricas de mayor varianza (con $k \leq 8$), y las variables ficticias se añadieron como paso directo. A modo orientativo, en *Airfoil Self-Noise* (5 numéricas, sin dummies) la longitud resultó de $5 \times 6 + \binom{5}{2} = 40$ genes; en *California Housing* (8 numéricas) de $8 \times 6 + \binom{8}{2} = 76$ genes.

Nota. En la presentación oral previa se indicó por error que el individuo tenía 5 atributos, pero la cifra correcta depende del dataset y del catálogo de transformaciones, como se dice en el texto anterior.

Comparadores externos. Para dar contexto al rendimiento del lineal con ingeniería de atributos, se añadieron dos modelos con sesgos inductivos distintos, K-Nearest Neighbors KNN, con estandarización previa y Random Forest. El primero captura relaciones locales no paramétricas y el segundo modela no linealidades mediante particiones de árboles agregadas. Ambos se entrenaron en el mismo split y se reportaron sus MAE/RMSE/R^2. Esta comparación permite interpretar si la representación $v(X)$ acerca al modelo lineal al desempeño de estimadores no lineales o, alternativamente, si el patrón del dato requiere mayor flexibilidad.

Salidas y trazabilidad. Se guardaron matrices optimizadas de entrenamiento y validación, la lista de transformaciones seleccionadas y un `JSON` con métricas y configuración de semilla, tamaños y generaciones. Para comunicación de resultados se generó un dashboard por conjunto de datos con (i) barra de RMSE por modelo y (ii) diagramas de dispersión $\hat{y}$ vs. y que ayudan a ver sesgos, saturaciones y dispersiones. La curva de fitness complementa el análisis al mostrar si la búsqueda se estancó pronto o si hubo mejoras progresivas.

4. RESULTADOS

Se evaluó el efecto de la transformación $v(\cdot)$ sobre dos conjuntos de datos de regresión, comparando el modelo lineal regularizado (RidgeCV) entrenado con $v(X)$ frente a dos referencias no lineales, *Random Forest* (RF) y k-vecinos ($k = 7$). La métrica de optimización fue MAE en validación y adicionalmente, se reportan RMSE y R^2 para complementar la lectura del error.

En *California Housing*, la aplicación de $v(X)$ redujo el MAE del lineal base de 0,533 a 0,438, lo que supone una mejora relativa del 17,9 %. En la comparación cruzada, RF obtuvo el menor RMSE (0,502) y el mayor R^2 (0,808), mientras que Ridge+$v(X)$ y KNN mostraron valores próximos entre sí (RMSE 0,647 y 0,654, $R^2 \approx 0{,}68$ y 0,67). El tablero con barras de RMSE y dispersión y vs. $\hat{y}$ se muestra en la Fig. 1 y la evolución del MAE por generación durante la búsqueda se observa en la Fig. 2. Las cifras resumen de MAE, RMSE y R^2 para cada modelo se recogen en la Tabla 1.

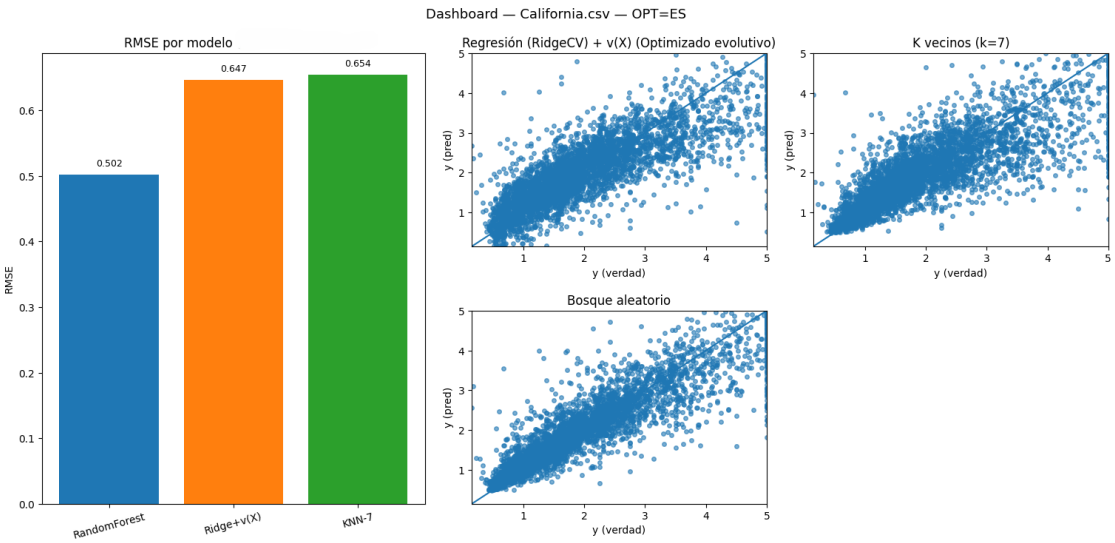

Fig. 1. Dashboard de modelos en *California*.

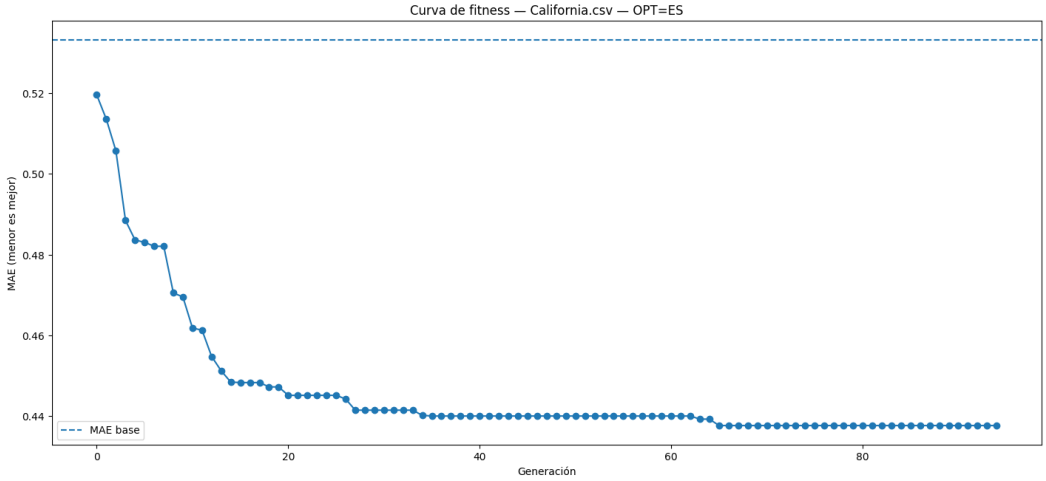

Fig. 2. Curva de fitness del optimizador (MAE por generación) en *California*.

TABLA 1. RESULTADOS EN *CALIFORNIA*.

Modelo	**MAE**	**RMSE**	R^2
Random Forest	0.325	0.502	0.808
KNN (k=7)	0.444	0.654	0.673
Ridge + $v(X)$	0.463	0.647	0.681

Fuente: ejecución propia.

En *Airfoil Self-Noise*, el MAE descendió de 3,672 (lineal base) a 2,962 con Ridge+$v(X)$, equivalente a una mejora relativa del 19,35 %. En la comparativa, RF lideró con RMSE 1,817 y $R^2 = 0{,}934$; KNN quedó intermedio (RMSE 3,628) y Ridge+$v(X)$ mostró RMSE 3,905 con $R^2 = 0{,}696$. El dashboard correspondiente se presenta en la Fig. 3, y los valores agregados se resumen en la Tabla 2.

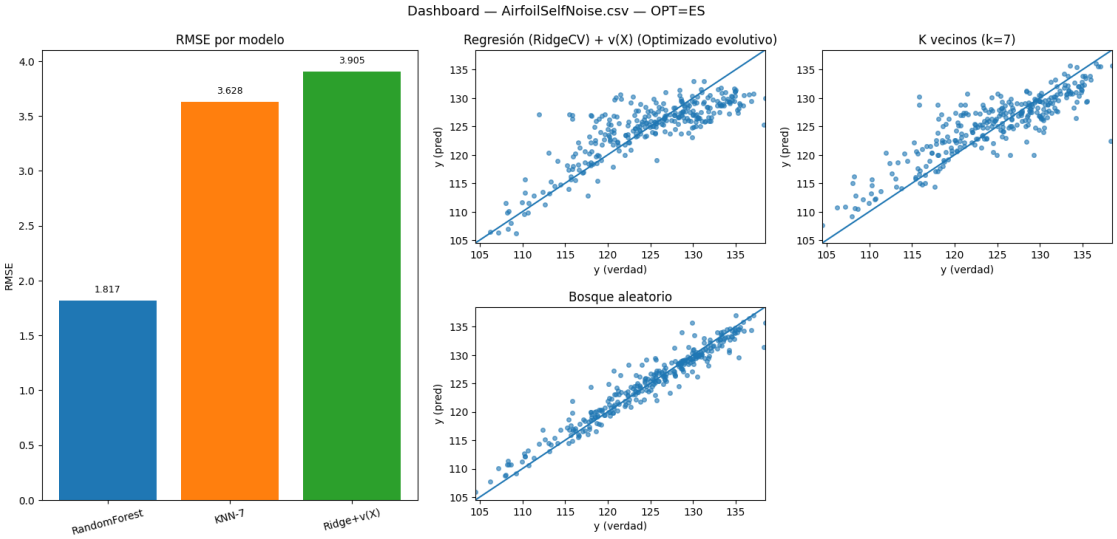

Fig. 3. Dashboard de modelos en *Airfoil Self-Noise*.

TABLA 2. RESULTADOS EN *AIRFOIL SELF-NOISE*.

Modelo	**MAE**	**RMSE**	R^2
Random Forest	1.303	1.817	0.934
KNN (k=7)	2.755	3.628	0.737
Ridge + $v(X)$	2.977	3.905	0.696

Fuente: ejecución propia.

En ambos conjuntos, los diagramas de dispersión del dashboard (Figs. 1 y 3) permiten comprobar la proximidad de las predicciones a la diagonal $y = \hat{y}$. En *California* se aprecia cómo Ridge+$v(X)$ corrige sesgos del lineal base y se acerca a KNN, mientras que RF mantiene una ligera ventaja en RMSE. En *Airfoil*, la ventaja de RF es más marcada, coherente con la presencia de no linealidades fuertes que las transformaciones univariadas e interacciones básicas no capturan del todo. La curva de *fitness* de *California* (Fig. 2) muestra caída temprana del MAE y meseta posterior, lo que sugiere convergencia temprana y ausencia de sobreajuste en la búsqueda bajo el *hold-out* empleado.

5. CONCLUSIONES

Para el trabajo se planteó el objetivo fue reducir el error absoluto medio (MAE) respecto al modelo lineal base, manteniendo un pipeline simple y reproducible.

En los dos conjuntos de regresión analizados se observó una reducción consistente del MAE del modelo lineal al aplicar $v(X)$. En *California Housing* la caída fue de 0,533 $\rightarrow$ 0,438 ($\approx$ 17,9 %), y en *Airfoil Self-Noise* de 3,672 $\rightarrow$ 2,962 ($\approx$ 19,35 %). En ambos casos, la transformación acercó el desempeño del lineal a modelos no lineales más expresivos. Aun así, vemos que *Random Forest* mantuvo el mejor RMSE y R^2, lo que sugiere que la relación subyacente presenta componentes no lineales, que el lineal incluso con ingeniería de atributos, no captura del todo.

Las curvas de fitness mostraron convergencia rápida: el MAE descendió bruscamente en las primeras generaciones y después se estabilizó, señal de que el operador evolutivo explora con eficacia el espacio de atributos y evita estancamientos prolongados.

En conjunto, puede afirmarse que el propósito del proyecto se cumple, ya que la transformación $v(\cdot)$ aprendida reduce el error del modelo lineal base y lo sitúa más cerca de referentes no lineales, ofreciendo una alternativa interpretable y de coste moderado.

Limitaciones y líneas de mejora

En la validación se empleó un único *hold-out*; sustituirlo por validación cruzada k-fold o anidada permitiría obtener estimaciones más estables y disminuir el riesgo de sobreajuste a una partición concreta.

La biblioteca de atributos utilizada incluyó transformaciones univariadas consideradas seguras, algunas interacciones y funciones trigonométricas normalizadas. Sería razonable ampliar el catálogo con funciones suaves como splines o *hinge*/piecewise, aplicar *binning* supervisado cuando proceda e incluso valorar transformaciones del objetivo, por ejemplo $\log y$, si el dominio lo justifica.

El criterio de optimización se centró únicamente en MAE. Un planteamiento multiobjetivo que equilibre error y parsimonia podría favorecer modelos más compactos y mejorar la interpretabilidad sin sacrificar rendimiento.

Finalmente, me resultaría conveniente estudiar la estabilidad de las columnas seleccionadas frente a cambios de semilla y de partición. Para ello se pueden incorporar técnicas de stability selection o regularización adicional que penalice soluciones inestables.

BIBLIOGRAFÍA

[1] A. Kumar, M. Boehm y J. Yang, "Data management in machine learning: Challenges, techniques, and systems," en *Proceedings of the ACM SIGMOD International Conference on Management of Data*, vol. Part F127746, Association for Computing Machinery, mayo de 2017, pp. 1717-1722. DOI: 10.1145/3035918.3054775.

[2] A. Kumar, J. Naughton y J. M. Patel, "Learning generalized linear models over normalized data," en *Proceedings of the ACM SIGMOD International Conference on Management of Data*, vol. 2015-May, Association for Computing Machinery, mayo de 2015, pp. 1969-1984. DOI: 10.1145/2723372.2723713.

[3] M. Mirmozaffari, N. A. Golilarz y S. S. Band, *Machine Learning Algorithms Based on an Optimization Model*, sep. de 2020. DOI: 10.20944/preprints202009.0729.v1. [En línea]. Disponible en: https://www.preprints.org/manuscript/202009.0729/v1.

6. ANEXOS

Anexo 1. Uso puntual de IA generativa

En este proyecto usé de forma puntual un asistente de IA, específicamente GPT5, para cosas muy concretas de código y formato, no para decidir la metodología completa del trabajo. Me sirvió sobre todo para ordenar los scripts (`run_single.py`, `vopt_core.py`), generar la barrita de progreso y la gráfica de fitness, y para arreglar detallitos cuando el preprocesado dejaba `NaN` o cuando alguna transformación regresaba `ndarray` sin `fillna`. También lo usé para que me diera una mano con LaTeX, citas IEEE con `biblatex`, cómo poner las figuras más grandes, y corregir alguna alerta de compilación. Las decisiones de fondo, incluyendo los datasets, métricas, comparación con RF y KNN, semillas, lectura de resultados, las tomé yo y las probé local.

El flujo fue más o menos así, yo corría los experimentos, veía el output, y cuando algo se atoraba o estaba repetitivo, pedía ayuda muy concreta. La IA proponía un fragmento, yo lo pegaba, ejecutaba y si no mejoraba o rompía algo, lo quitaba. Todo lo que está en resultados (MAE, RMSE, R^2) sale de mi entorno con `scikit-learn`, y las figuras de dashboards y fitness se generaron con la versión final que entrego aquí. Fijé semillas para que sea reproducible y dejé el *hold-out* explícito.

Dejo algunos prompts reales/parecidos a lo que usé (para que se entienda el tipo de ayuda). Son más bien pedazos de conversación de cuando me atascaba:

> "Alguna forma ligera de registrar la mejor MAE por generación y graficarla sin meter dependencias nuevas?"
> "Necesito un *helper* para asegurar que las transformaciones devuelvan `Series` alineadas aunque alguna regrese `ndarray`."
> "Estoy escribiendo en LaTeXcómo puedo poner las figuras a ancho completo y que el pie quede limpio?"
> "me marca error: AttributeError: 'numpy.ndarray' object has no attribute fillna ¿qué cambio?"
> "¿Qué parámetros puedo dejar arriba para explorar sin convertir esto en una búsqueda masiva?"

La IA me ahorró tiempo en tareas mecánicas como refactor, visualización, LaTeX, y me dio ideas para depurar más rápido, pero el diseño del experimento, la comparación con los baselines, y la interpretación de los resultados los hice yo con lo visto en clase y las pruebas que corrí.